Smart Contract Deployment on Ethereum
以太坊智能合約部署與 DApp 實作

Technical Report
技術報告

Author 作者：劉子敬
Department：國立高雄師範大學 物理學系

一、專題介紹

本專題實作如何在 Ethereum 測試網路上部署一個智能合約，及一個去中心化應用程式（Decentralized Application, DApp），透過 Ethereum 區塊鏈實現一個簡單的留言板系統。使用者可以將訊息寫入區塊鏈，並從區塊鏈讀取目前儲存的訊息。此專題展示 Web3 應用程式的完整流程，包括：智能合約撰寫、區塊鏈部署、錢包互動以及前端網頁整合。

二、系統架構

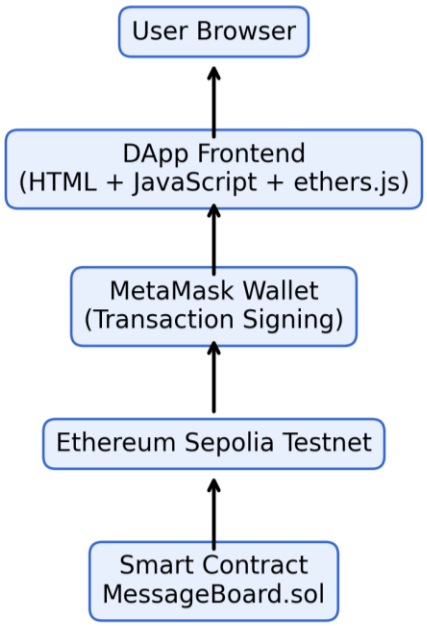
本系統由四個主要部分組成：

1. 使用者瀏覽器
2. MetaMask 錢包
3. Ethereum Sepolia 測試網
4. Solidity 智能合約

前端網頁透過 ethers.js 與 MetaMask 溝通，MetaMask 負責簽署交易並將交易送至 Ethereum 網路。

三、系統架構圖

下圖展示 Web3 應用的系統架構：



四、使用技術

區塊鏈平台：Ethereum
測試網路：Sepolia Testnet
智能合約語言：Solidity 0.8.x
開發工具：Remix IDE
錢包：MetaMask
前端技術：HTML、JavaScript、ethers.js

五、智能合約設計

本專題的智能合約 MessageBoard.sol 用來儲存區塊鏈上的留言。合約包含 message 變數儲存目前留言，以及 owner 紀錄部署者地址。使用者可以呼叫 setMessage() 更新留言，也可以透過 getMessage() 讀取目前的留言內容。

- setMessage(string memory _msg)：更新區塊鏈上的留言
- getMessage()：讀取區塊鏈上的留言

六、智能合約部署流程

本專題使用 Remix IDE 進行合約部署。主要步驟如下：

1. 開啟 Remix IDE。
2. 建立 Solidity 檔案 MessageBoard.sol。
3. 使用 Solidity 0.8.x 編譯合約。
4. 在 Deploy & Run 中選擇 Injected Provider – MetaMask。
5. 切換 MetaMask 至 Sepolia Testnet。
6. 點擊 Deploy 並在 MetaMask 中確認交易。

七、區塊鏈交易流程

在 Ethereum 區塊鏈中，每一筆交易都需要支付 Gas。Gas 是用來補償節點執行智能合約所消耗的計算資源。

當部署合約或呼叫 setMessage 時，交易會包含：
- Transaction Hash
- Gas Fee
- Block Number
- Sender Address
- Contract Address

交易被區塊鏈確認後，智能合約狀態即被更新

當使用者在網頁輸入新的留言並按下 Send Transaction，前端會呼叫智能合約的 setMessage 函數。MetaMask 會跳出交易確認視窗，使用者確認後交易被送至 Ethereum 網路。交易被礦工或驗證節點打包進區塊後，區塊鏈狀態即被更新。

八、DApp 前端介面

本專題額外建立一個簡單的前端網頁，使使用者能透過 MetaMask 與合約互動。前端使用 ethers.js 讀取與寫入區塊鏈資料。

1. 連接 MetaMask 錢包
2. 從區塊鏈讀取目前留言
3. 發送交易更新留言
4. 在 Etherscan 查看交易紀錄

九、實驗結果

實驗結果顯示，使用者可以成功透過網頁介面更新區塊鏈留言。交易經由 MetaMask 確認後被寫入 Sepolia 測試網，並可在 Etherscan 上查看交易紀錄。重新讀取訊息時，前端成功顯示區塊鏈上的最新留言。

十、系統畫面

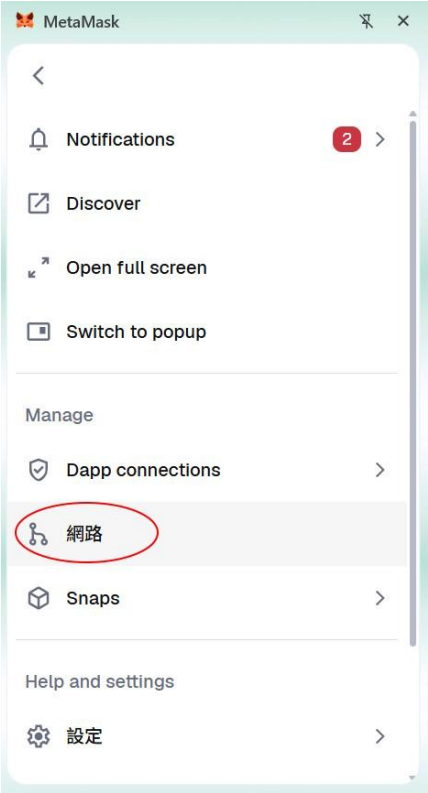


Figure 1: 安裝 MetaMask 並選擇網路

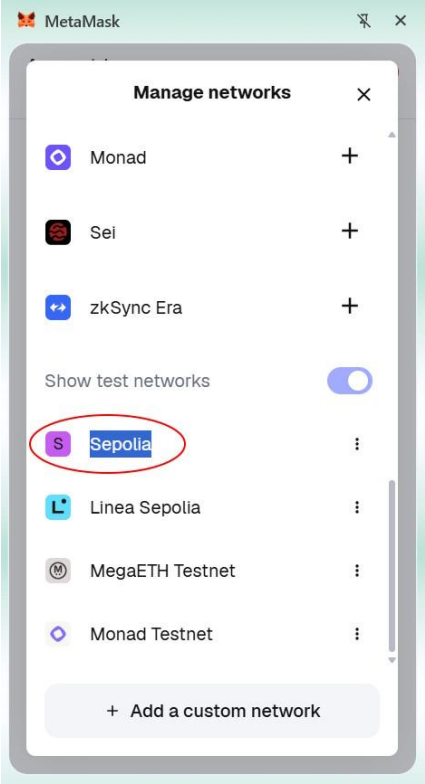


Figure 2: 使用測試網 Sepolia，不是乙大坊主網

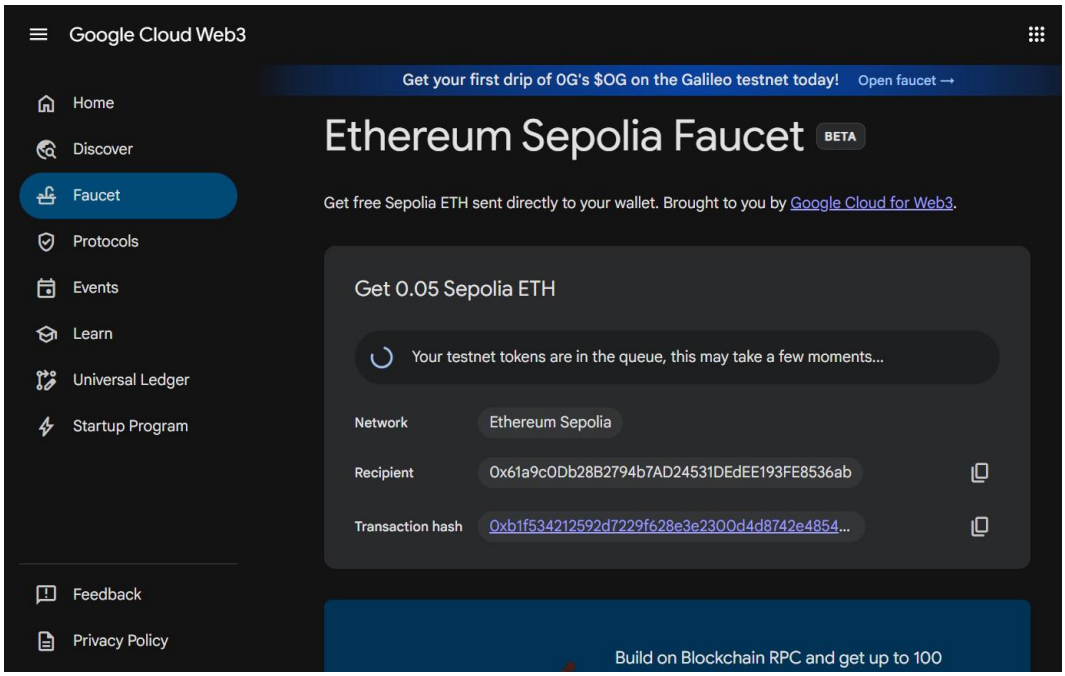


Figure 3: 從 Google Cloud Web3 取得測試乙太幣 (用以部署合約)

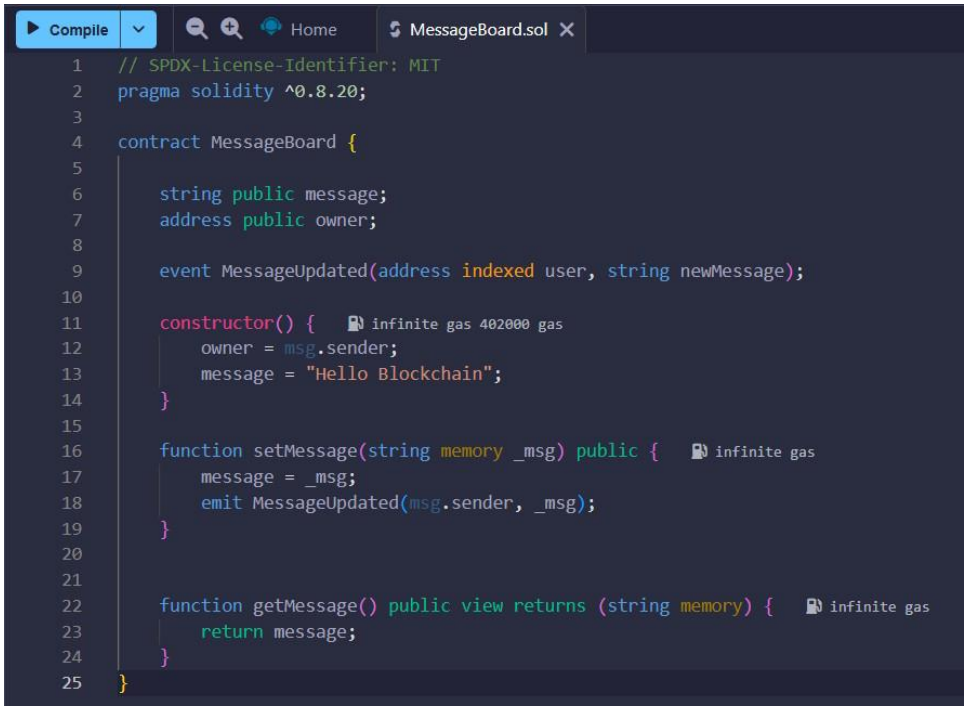


Figure 4: 用 RemixIDE 進行開發

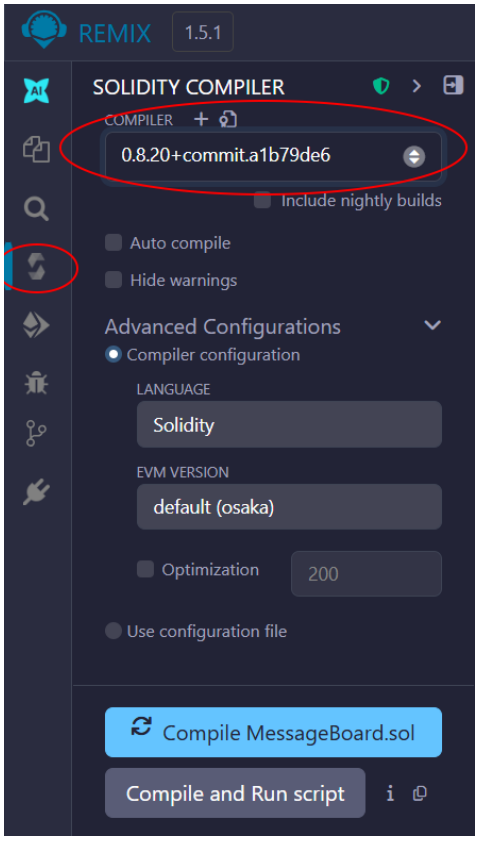


Figure 5: 編譯器版本選擇

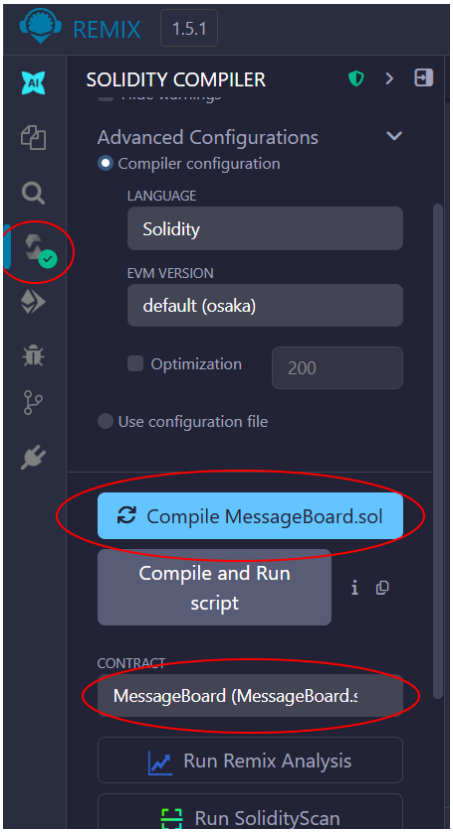


Figure 6: 進行編譯

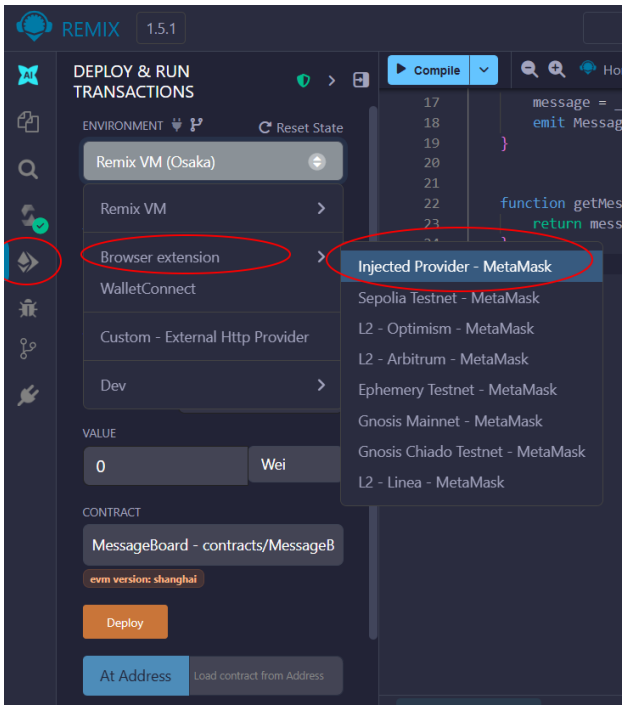


Figure 7: 部署時連上錢包 MetaMask

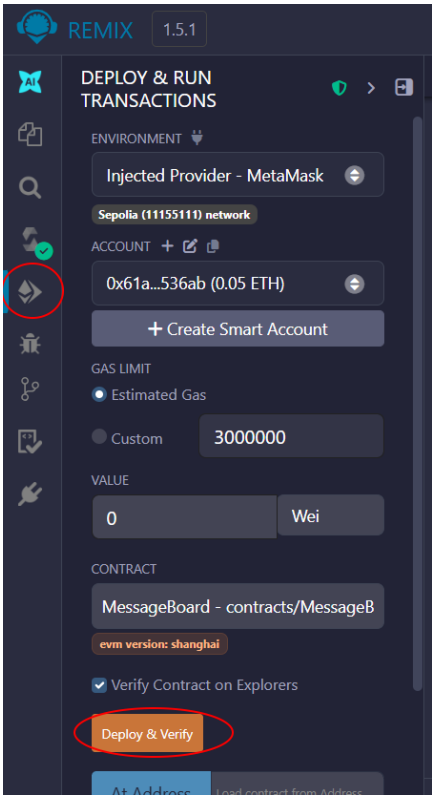


Figure 8: 進行部署(及驗證)

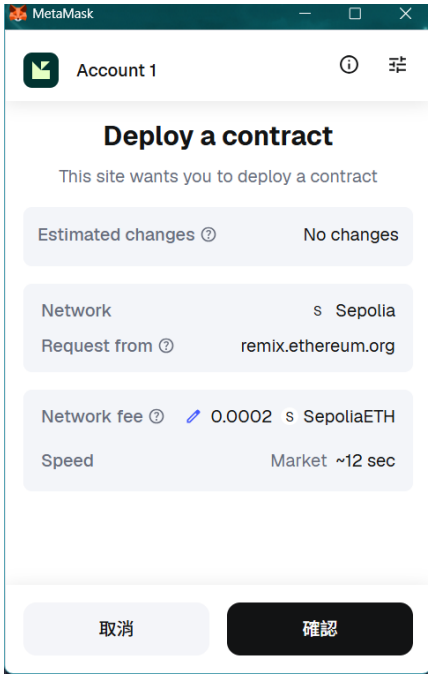


Figure 9: MetaMask 錢包的確認

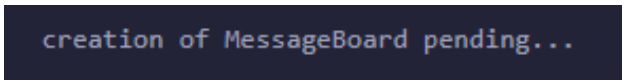


Figure 10: 緊張的等待

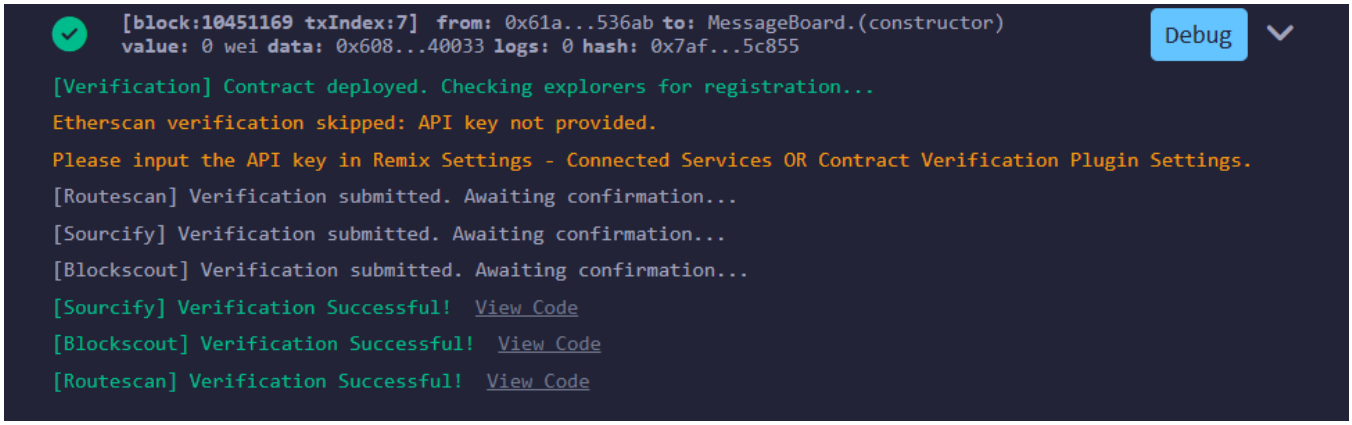


Figure 11 : RemixIDE 訊息欄顯示驗證完成！

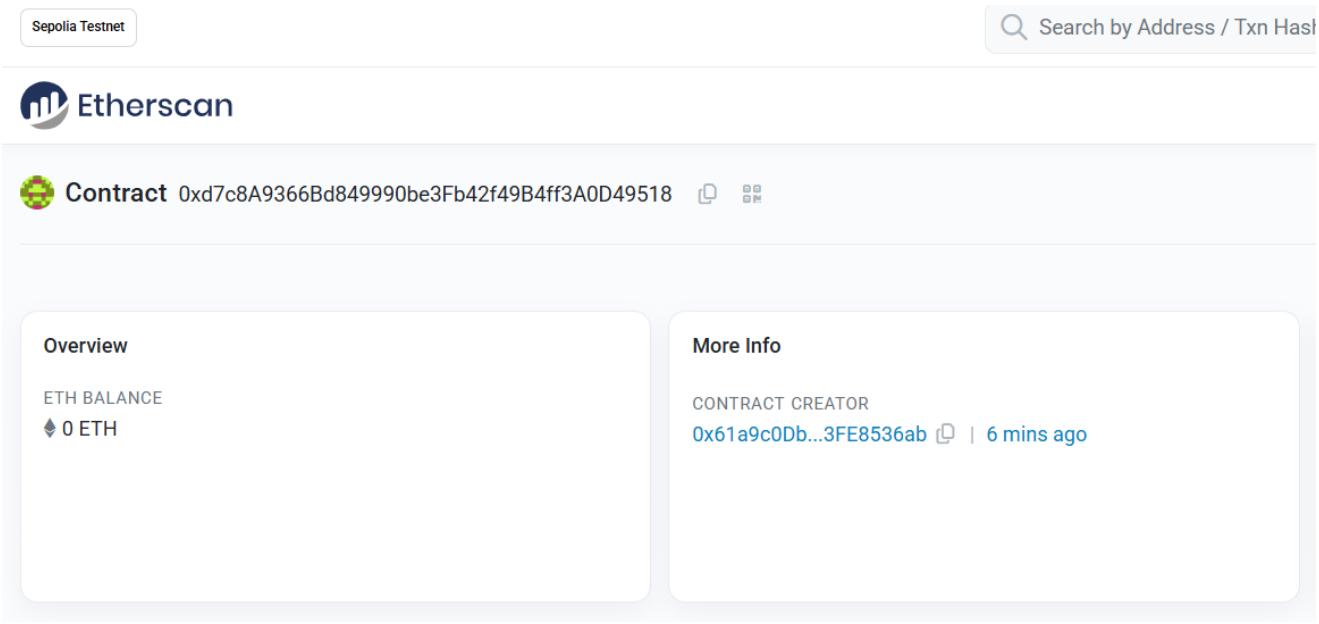


Figure 12 : 鏈上 Etherscan 也能看到合約了！ (是得用測試網的 Etherscan， 不是主網的)

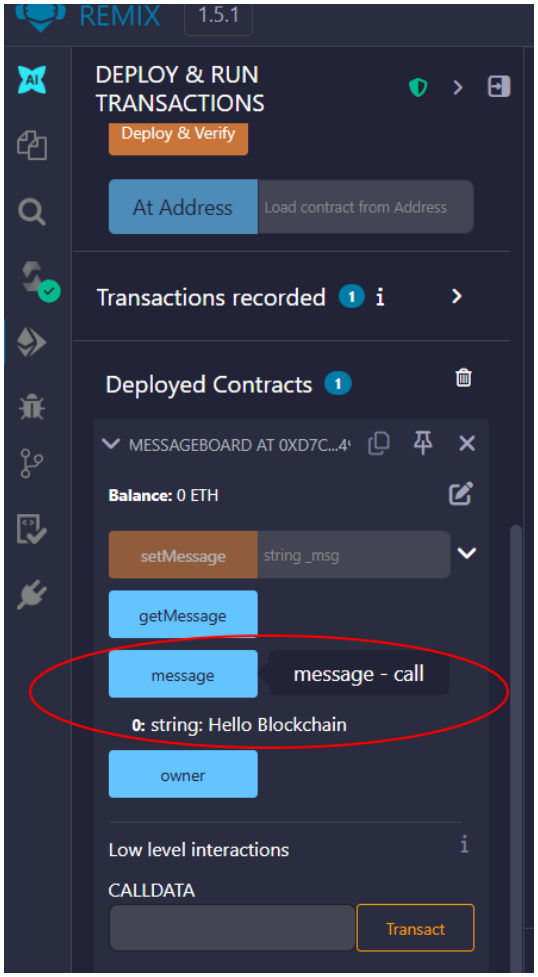


Figure 13: RemixIDE 中即可讀取

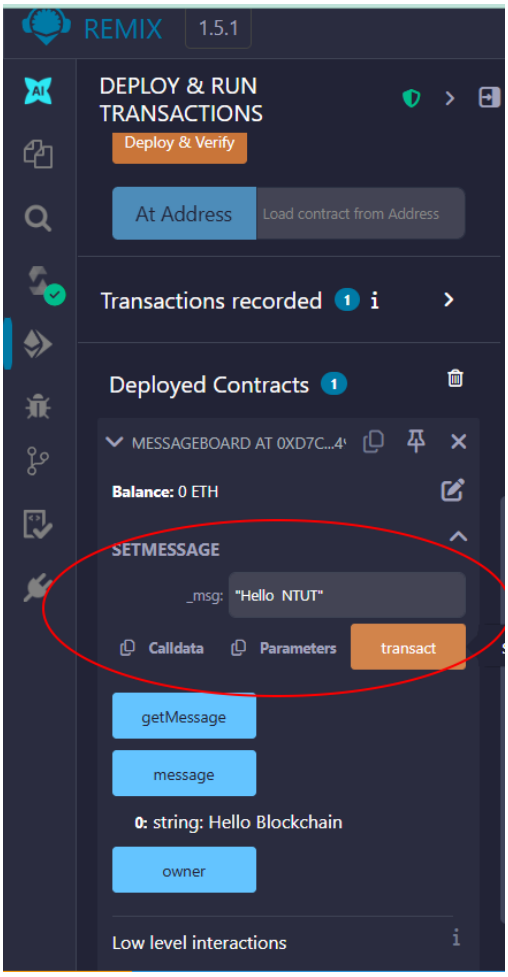


Figure 14: SetMessage 為新訊息

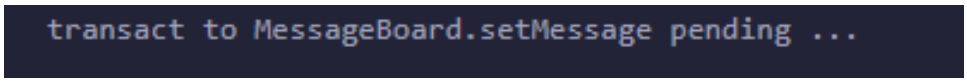


Figure 15: 交易等待中

Figure 16: 交易成功！

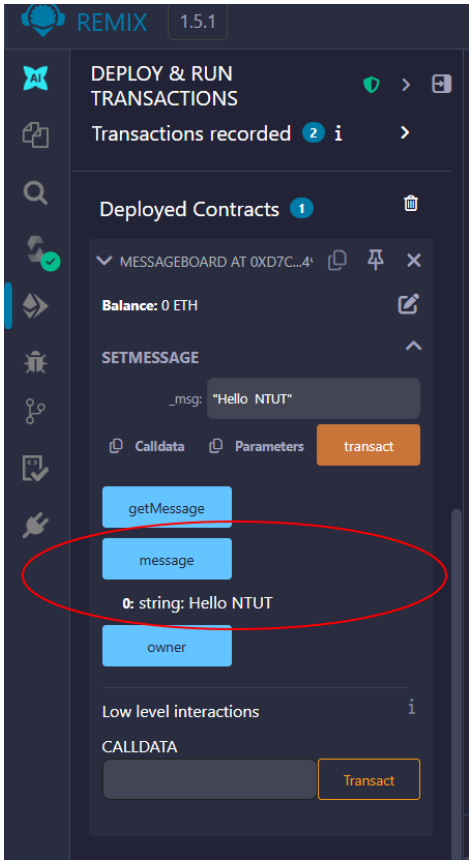


Figure 17: 檢查 - 鏈上訊息更新了

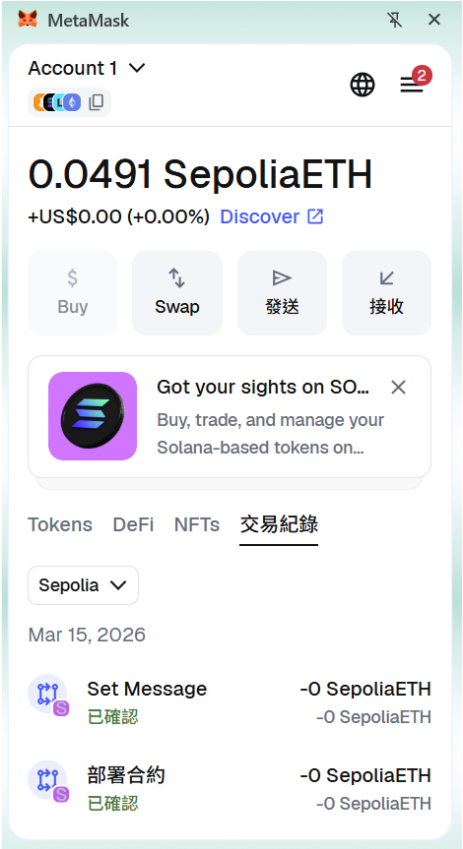


Figure 18: MetaMask 中也有紀錄

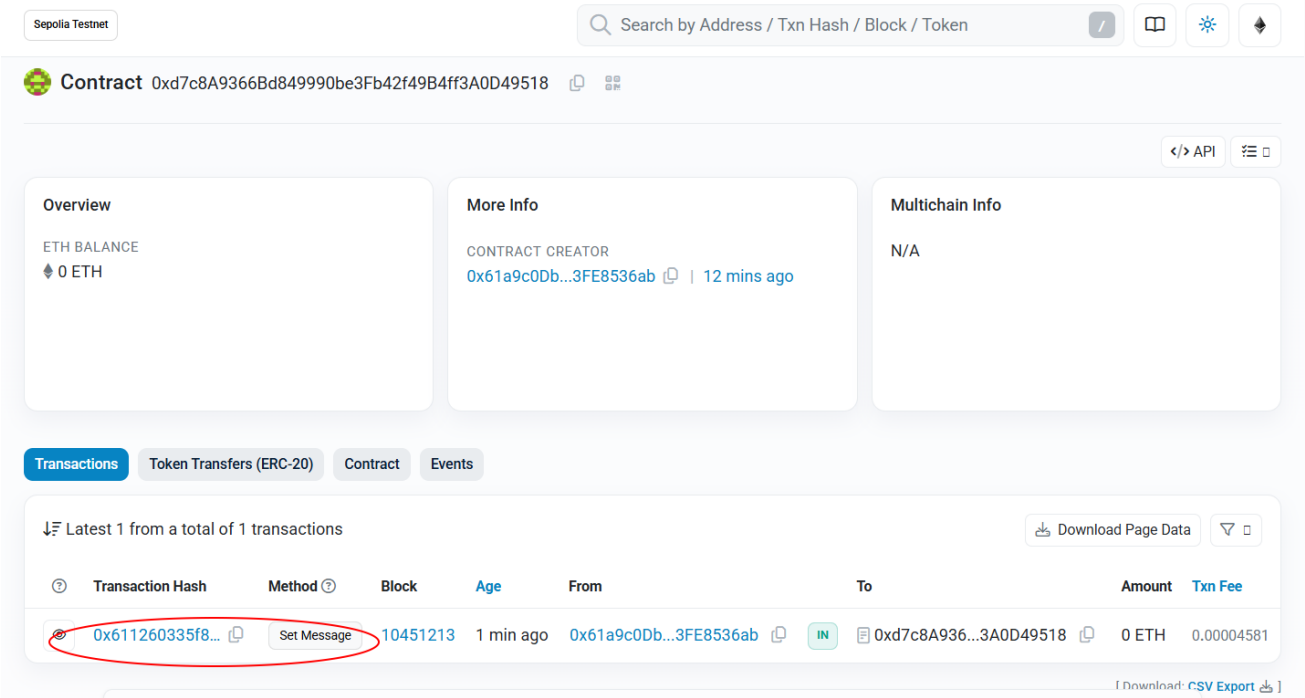


Figure 19: Etherscan 可以查到這個 Transcation

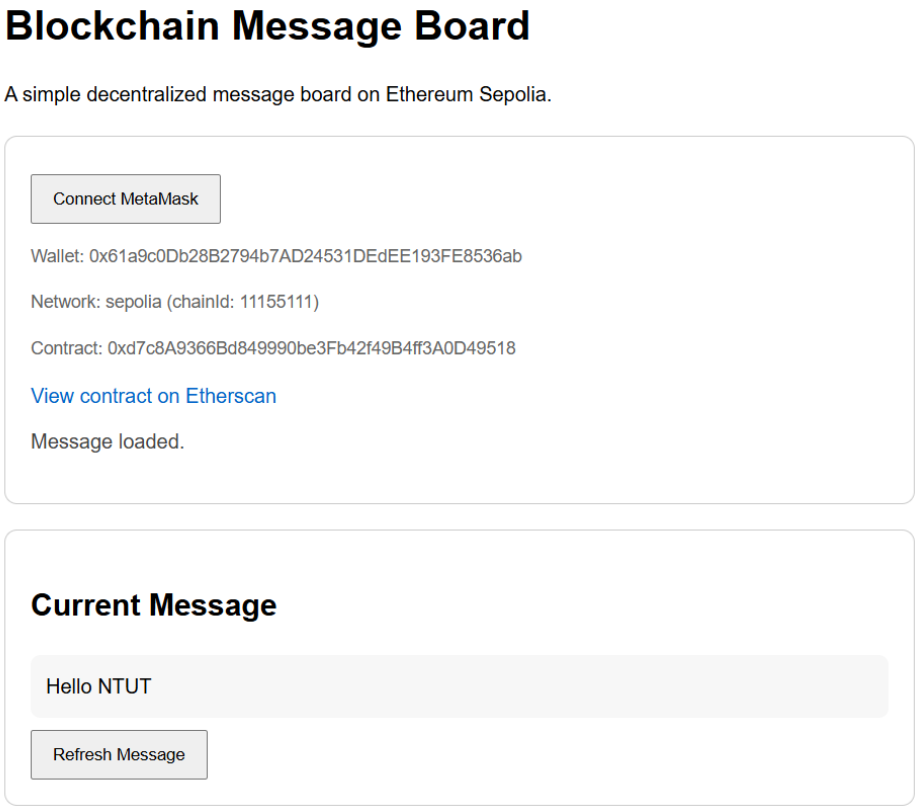


Figure 20: 不滿足於 RemixIDE，也來實作了簡單的 DApp。使用時先要連接到 MetaMask 。會顯示出現有的訊息。

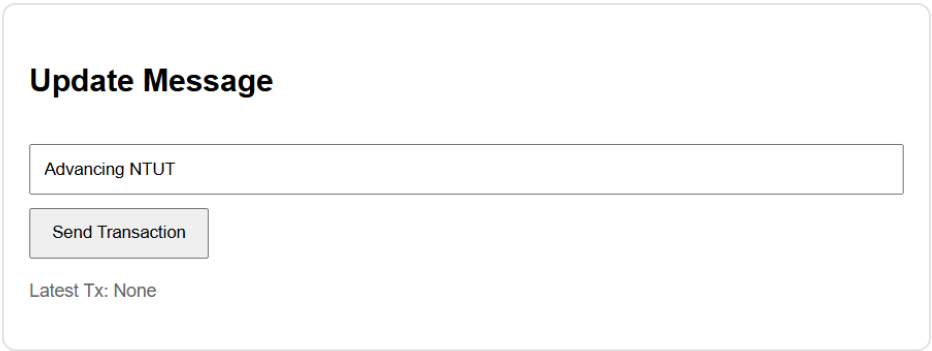


Figure21: 也可以再更新訊息！

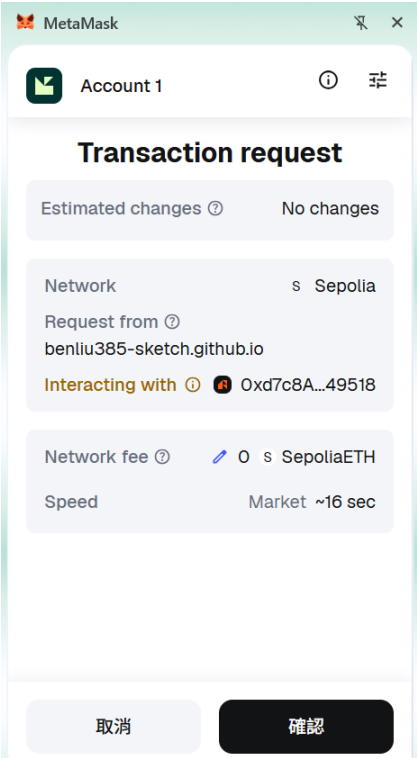


Figure 22: 一樣會觸發 MetaMask 確認交易

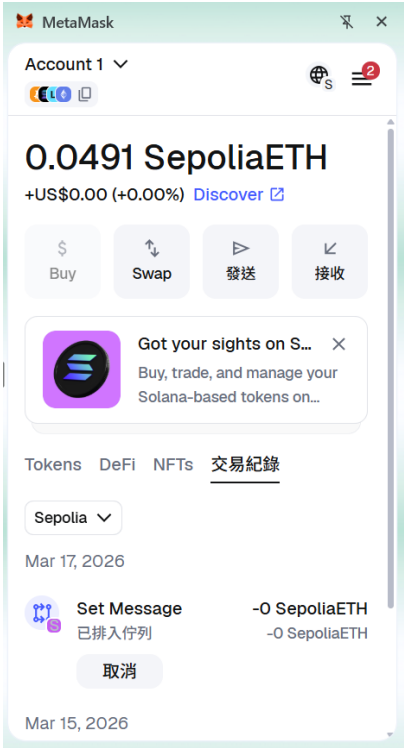


Figure 23: 確認完排入佇列

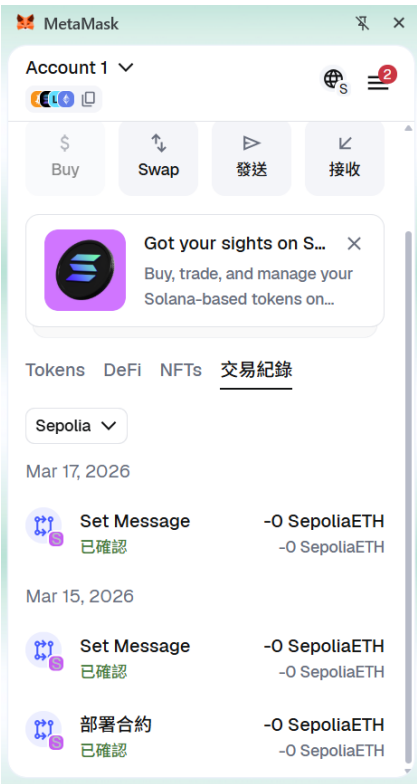


Figure 24: 已確認上鏈

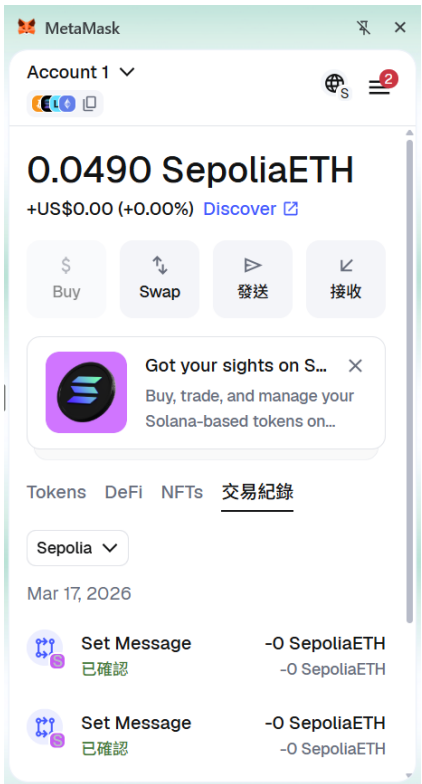


Figure 25: 錢包金額也減少了

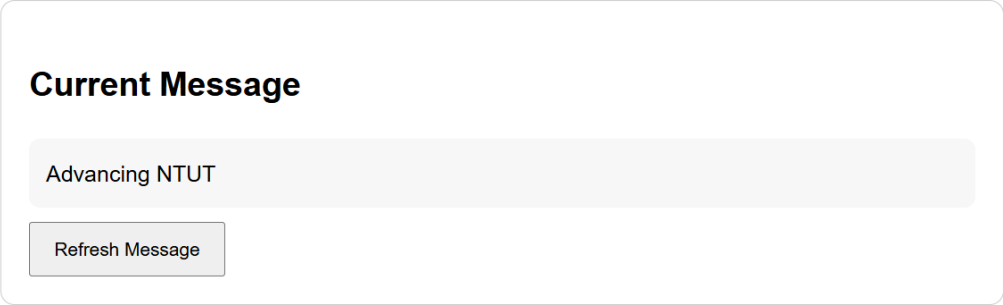


Figure 26: 最後確認，真的讀出新的訊息！

十一、 學習成果

透過本專題，我們學習到：
1. Solidity 智能合約撰寫
2. 使用 Remix 部署智能合約
3. 使用 MetaMask 與區塊鏈互動
4. 使用 ethers.js 建立 Web3 前端應用
5. 使用 Etherscan 驗證交易

十二、 未來發展

未來可以進一步擴展此專題，例如：
- 多人留言系統
- 完整 Web3 前端框架
- 支援多筆留言
- 建立留言歷史紀錄
- 部署至 Ethereum 主網
- 建立完整去中心化應用系統